

07

# Lecture 07

**CPU Scheduling Algorithms I: FCFS and SJF**

by - Brijesh Shukla  
C07: Operating Systems

**Join the lecture online on your dashboard.**

# Why CPU Scheduling is Essential :

## The Reality:

Your laptop right now has:

- **200+ processes** competing for CPU
- **Only 4-8 cores** to run them
- **Some processes take milliseconds, others take hours**

## Without Scheduling:

- Opening a text file waits for video encoding
- Mouse clicks lag during software updates
- System tasks can't interrupt user programs
- Long processes block everything

## With Smart Scheduling:

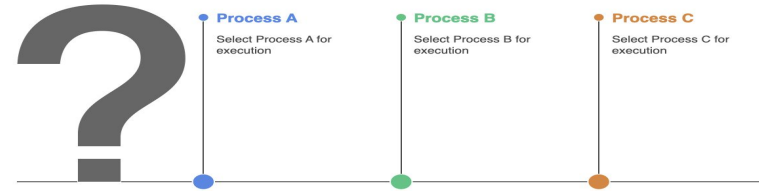
-  Responsive user interface
-  Fair resource distribution
-  Optimized waiting times
-  Predictable system behavior

# CPU Scheduling – The Three Ws (Where, Who, When):

Where done?

Ready Queue

Which process should be selected from the Ready Queue for CPU allocation?



## Short-Term Scheduler Process



Who did it?

Short-Term Scheduler (or CPU Scheduler).

When?

Scheduling Criterion (Preemptive or Non-Preemptive)

What type of scheduling criterion should be used?

### Non-Preemptive Scheduling

Processes run to completion without interruption, maintaining stability.



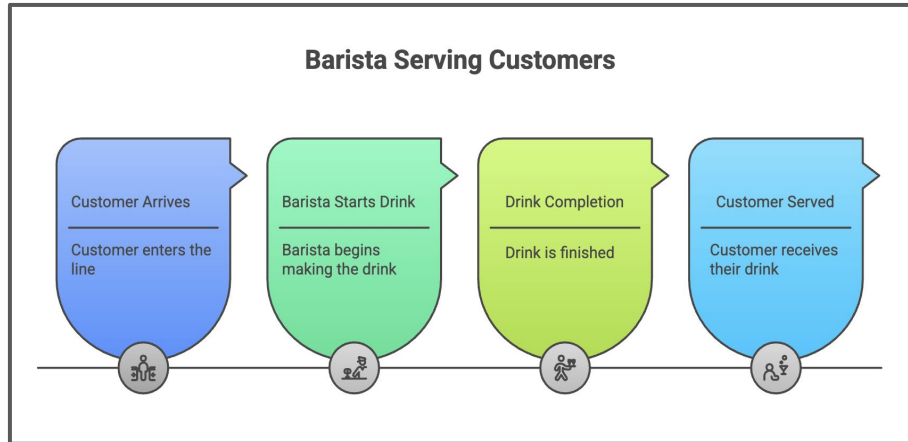
### Preemptive Scheduling

Allows interruption of processes for higher priority tasks, ensuring responsiveness.

# The Coffee Shop Analogy:

Imagine a single Barista trying to serve a line of Customers.

- **The Goal:** Serve everyone as fast as possible.
- **The Constraint:** The Barista can only make one drink at a time.



# The Coffee Shop Analogy: Three customers arrive at a coffee shop

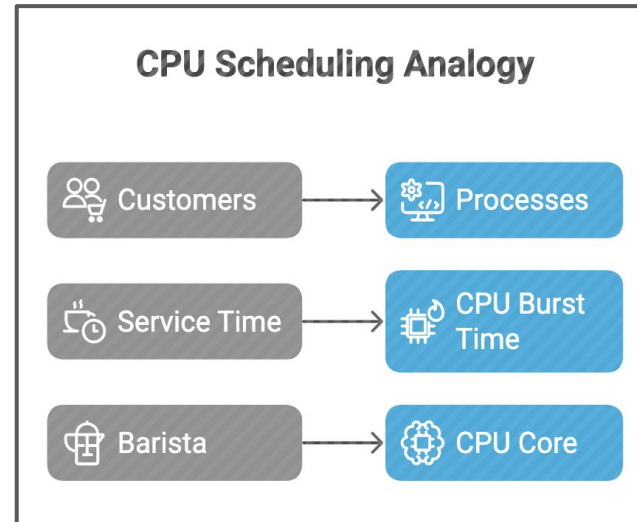
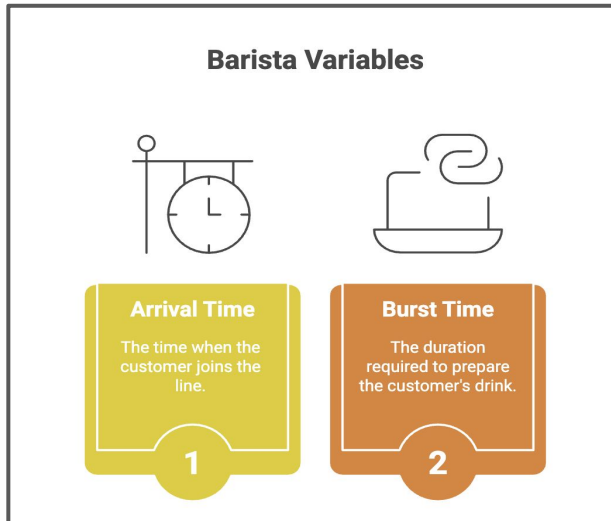
**Customer A (arrives at 9:00 AM)** → Orders 10 lattes → Takes 10 minutes

**Customer B (arrives at 9:01 AM)** → Orders 1 espresso → Takes 1 minute

**Customer C (arrives at 9:02 AM)** → Orders 1 cappuccino → Takes 1 minute

Question: What's the fairest way to serve them?

➡ This is CPU Scheduling!



# The Coffee Shop Analogy:

## The Scheduling Challenge:

-  1 CPU (your single barista)
-  Multiple processes (customer orders)
-  Different execution times
-  Goal: Minimize waiting time

## Which service strategy should be implemented?

**SJF**  
Reduces average  
wait time by serving  
fastest orders first.

**SRTF**

Optimizes efficiency  
by always serving  
the quickest  
remaining order.

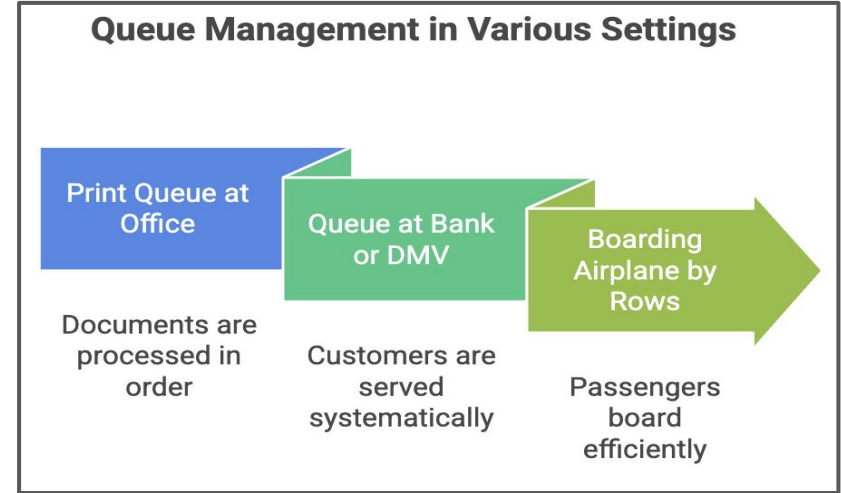
**FCFS**

Ensures fairness by  
serving customers in  
arrival order.

# FCFS (First Come First Served) - The "Wait Your Turn" Algorithm:

## Core Principle:

- Serve processes in exact arrival order
- No cutting in line allowed
- Once started, process runs to completion



## Mechanism:

- **Criteria:** Arrival Time only ( will be the Execution order ).
- **Mode:** Non-Preemptive (Once the smoothie starts blending, the Barista won't stop).
- **Data Structure:** FIFO Queue (First-In, First-Out).



## FCFS (First Come First Served) - The "Wait Your Turn" Algorithm:

### FCFS Algorithm Process:

**Step 1:** Maintain a Ready Queue (FIFO)

**Step 2:** When CPU is free, pick process at head of queue

**Step 3:** Execute process until completion (non-preemptive)

**Step 4:** Remove from queue, pick next process

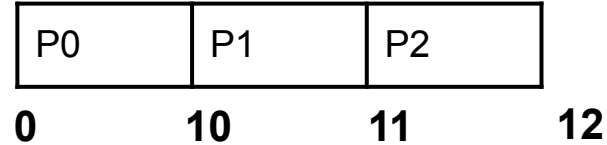
**Step 5:** Repeat

| Process Timing Acronyms |     |
|-------------------------|-----|
| Process Id              | Pid |
| Arrival Time            | AT  |
| Burst Time              | BT  |
| Turnaround Time         | TAT |
| Waiting Time            | WT  |
| Average Waiting Time    | AWT |

## Example 1 - FCFS:

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 1       | 1       |
| P2  | 2       | 1       |

### Gantt Chart



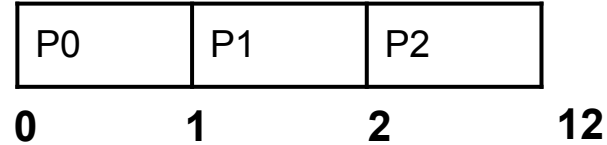
| Pid | AT | BT | CT | TAT = CT-AT | WT = TAT - BT |
|-----|----|----|----|-------------|---------------|
| P0  | 0  | 10 | 10 | 10 - 0 = 10 | 10 - 10 = 0   |
| P1  | 1  | 1  | 11 | 11 - 1 = 10 | 10 - 1 = 9    |
| P2  | 2  | 1  | 12 | 12 - 2 = 10 | 10 - 1 = 9    |

$$AWT = \sum WT / \text{Total Number of Process} = (0 + 9 + 9) / 3 = 6 \text{ ms}$$

## Example 2 - FCFS:

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 1       |
| P1  | 1       | 1       |
| P2  | 2       | 10      |

### Gantt Chart



| Pid | AT | BT | CT | TAT = CT-AT   | WT = TAT - BT |
|-----|----|----|----|---------------|---------------|
| P0  | 0  | 1  | 1  | $1 - 0 = 1$   | $1 - 1 = 0$   |
| P1  | 1  | 1  | 2  | $2 - 1 = 1$   | $1 - 1 = 0$   |
| P2  | 2  | 10 | 12 | $12 - 2 = 10$ | $10 - 10 = 0$ |

$$AWT = (0 + 0 + 0) / 3 = 0 \text{ ms}$$

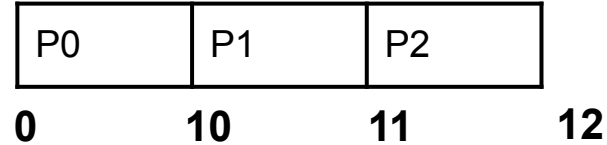
### Example 3 - FCFS - What if ?

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 0       | 1       |
| P2  | 0       | 1       |

### Example 3 - FCFS - What if ?

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 0       | 1       |
| P2  | 0       | 1       |

**Gantt Chart**



| Pid | AT | BT | CT | TAT = CT-AT | WT = TAT - BT |
|-----|----|----|----|-------------|---------------|
| P0  | 0  | 10 | 10 | 10 - 0 = 10 | 10 - 10 = 0   |
| P1  | 0  | 1  | 11 | 11 - 0 = 11 | 11 - 1 = 10   |
| P2  | 0  | 1  | 12 | 12 - 0 = 12 | 12 - 1 = 11   |

$$AWT = ( 0 + 10 + 11 ) / 3 = 7 \text{ ms}$$

## FCFS - The Convoy Effect Problem:

### In CPU Scheduling:

- One long process holds up many short processes
- Average waiting time increases drastically
- Poor for interactive/responsive systems

#### Example 1:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 1  | 2  |
| BT  | 10 | 1  | 1  |

$$\text{AWT} = ( 0 + 9 + 9 ) / 3 \\ = 6 \text{ ms}$$

#### Example 2:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 1  | 2  |
| BT  | 1  | 1  | 10 |

$$\text{AWT} = ( 0 + 0 + 0 ) / 3 \\ = 0 \text{ ms}$$

#### Example 3:

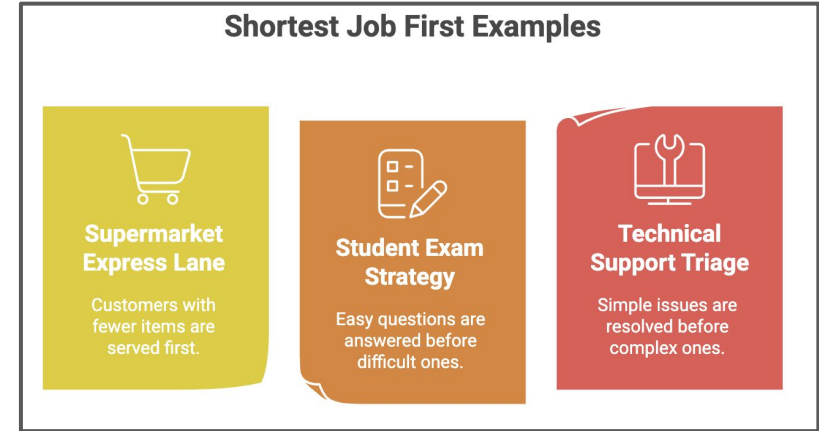
| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 0  | 0  |
| BT  | 10 | 1  | 1  |

$$\text{AWT} = ( 0 + 10 + 11 ) / 3 \\ = 7 \text{ ms}$$

# SJF (Shortest Job First) - The "Non Preemptive Minimum Execution Time" Algorithm:

## Core Principle:

- Select process with the shortest next CPU burst
- Shortest tasks cut to the front of the line
- Once started, process runs to completion
- Provably optimal for minimizing average waiting time



## Mechanism:

- **Criteria:** Burst Time (Smallest first).
- **Mode:** Non-Preemptive (Wait for the current drink to finish, then pick the shortest next).
- **Data Structure:** Priority Queue (Min-Heap) data structure.

## SJF (Shortest Job First) - The "Non Preemptive Minimum Execution Time" Algorithm:

### SJF Algorithm Process:

**Step 1:** Maintain priority queue sorted by burst time

**Step 2:** When CPU is free, select SHORTEST burst

**Step 3:** Execute selected process to completion

**Step 4:** Remove from queue, reselect shortest

**Step 5:** Repeat

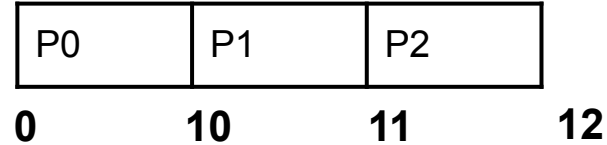
| Process Timing Acronyms |     |
|-------------------------|-----|
| Process Id              | Pid |
| Arrival Time            | AT  |
| Burst Time              | BT  |
| Turnaround Time         | TAT |
| Waiting Time            | WT  |
| Average Waiting Time    | AWT |



## Example 1 - SJF:

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 1       | 1       |
| P2  | 2       | 1       |

### Gantt Chart



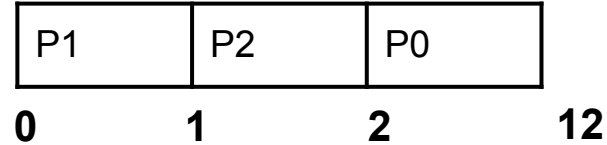
| Pid | AT | BT | CT | TAT = CT-AT   | WT = TAT - BT |
|-----|----|----|----|---------------|---------------|
| P0  | 0  | 10 | 10 | $10 - 0 = 10$ | $10 - 10 = 0$ |
| P1  | 1  | 1  | 11 | $11 - 1 = 10$ | $10 - 1 = 9$  |
| P2  | 2  | 1  | 12 | $12 - 2 = 10$ | $10 - 1 = 9$  |

$$AWT = ( 0 + 9 + 9 ) / 3 = 6 \text{ ms}$$

## Example 2 - SJF - What if ?

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 0       | 1       |
| P2  | 0       | 1       |

**Gantt Chart**



| Pid | AT | BT | CT | TAT = CT-AT   | WT = TAT - BT |
|-----|----|----|----|---------------|---------------|
| P0  | 0  | 10 | 12 | $12 - 0 = 12$ | $12 - 10 = 2$ |
| P1  | 0  | 1  | 1  | $1 - 0 = 1$   | $1 - 1 = 0$   |
| P2  | 0  | 1  | 2  | $2 - 0 = 2$   | $2 - 1 = 1$   |

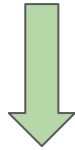
$$AWT = ( 2 + 0 + 1 ) / 3 = 1 \text{ ms}$$

## SJF - The Convoy Effect Problem:

### Example 1:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 1  | 2  |
| BT  | 10 | 1  | 1  |

$$AWT = ( 0 + 9 + 9 ) / 3 = 6 \text{ ms}$$

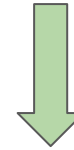


The **Convoy Effect is present here**: because the longest job is executing first, forcing shorter jobs to wait, which drastically increases the average waiting time to 6 ms.

### Example 2:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 0  | 0  |
| BT  | 10 | 1  | 1  |

$$AWT = ( 2 + 0 + 1 ) / 3 = 1 \text{ ms}$$



The **Convoy Effect is not present here**: because the shortest jobs are executing first, allowing them to finish quickly, which drastically decreases the average waiting time to 1 ms.

## SJF - Starvation Problem:

| Initial State                     |         |         |
|-----------------------------------|---------|---------|
| Pid                               | AT (ms) | BT (ms) |
| P0                                | 0       | 10      |
| P1                                | 0       | 12      |
| P2                                | 2       | 1       |
| P3                                | 4       | 100     |
| Continuous Stream of Shorter Jobs |         |         |
| P4                                | 8       | 5       |
| P5                                | 10      | 7       |
| ▪                                 | ▪       | ▪       |
| ▪                                 | ▪       | ▪       |



In this example, **P3 (100 ms)** is the victim of starvation. Even though it arrived at Time 4, it is indefinitely blocked by **P4, P5....** and any future short processes that enter the system."

## SJF - What if ?

| Initial State                     |         |         |
|-----------------------------------|---------|---------|
| Pid                               | AT (ms) | BT (ms) |
| P0                                | 0       | 100     |
| P1                                | 0       | 12      |
| P2                                | 2       | 1       |
| P3                                | 4       | 10      |
| Continuous Stream of Shorter Jobs |         |         |
| P4                                | 8       | 5       |
| P5                                | 10      | 7       |
| ▪                                 | ▪       | ▪       |
| ▪                                 | ▪       | ▪       |

## Starvation ?



**Convey Effect will be there.**

## Problems of SJF :

- Not Responsive Enough
- Convey Effect still exists
- Problem of Starvation

# SRTF (Shortest Remaining Time First) - The "Preemptive Minimum Remaining Time" Algorithm

## Core Principle:

- Select process with the shortest remaining time
- Check at EVERY arrival and completion
- Preempt if new process is shorter
- Provably optimal for minimizing average waiting time

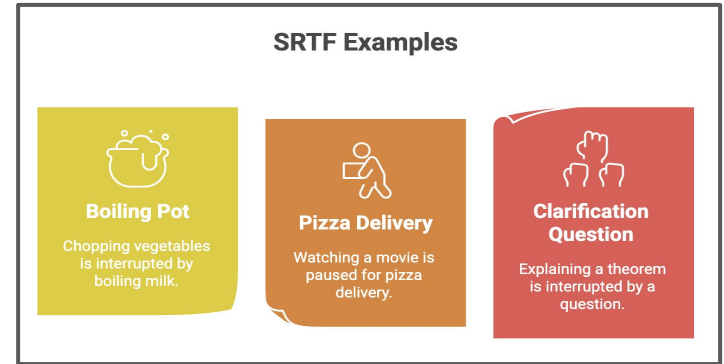
## Mechanism:

- **Criteria:** Remaining Burst Time (Shortest remaining first).
- **Mode:** Preemptive (If a new job arrives with less time than what is left on the current one, stop and switch).
- **Data Structure:** Priority Queue data structure.

## Note:

**SJF:** Decisions happen only when a process **finishes**.

**SRTF:** Decisions happen when a process **finishes** OR when a new process **arrives**.



## SRTF - The "Preemptive Minimum Remaining Time" Algorithm:

### SRTF Algorithm Process:

At every event (arrival or completion):

**Step 1:** Calculate remaining time for ALL ready processes

**Step 2:** Select process with SHORTEST remaining time

**Step 3:** Is it different from currently running process?

→ YES: Preempt (context switch)

→ NO: Continue current process

**Step 4:** Execute for 1 time unit (or until next event)

**Step 5:** Repeat

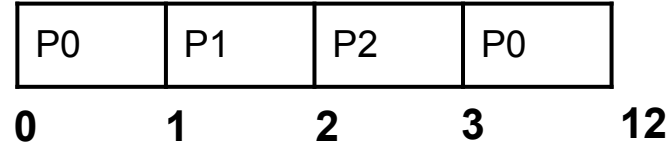
| Process Timing Acronyms |     |
|-------------------------|-----|
| Process Id              | Pid |
| Arrival Time            | AT  |
| Burst Time              | BT  |
| Turnaround Time         | TAT |
| Waiting Time            | WT  |
| Average Waiting Time    | AWT |



## Example 1 - SRTF:

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 1       | 1       |
| P2  | 2       | 1       |

### Gantt Chart



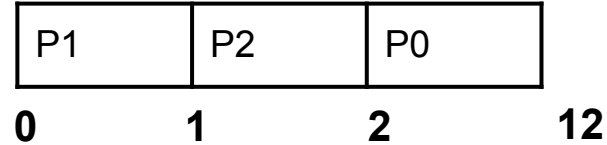
| Pid | AT | BT | CT | TAT = CT-AT   | WT = TAT - BT |
|-----|----|----|----|---------------|---------------|
| P0  | 0  | 10 | 12 | $12 - 0 = 12$ | $12 - 10 = 2$ |
| P1  | 1  | 1  | 2  | $2 - 1 = 1$   | $1 - 1 = 0$   |
| P2  | 2  | 1  | 3  | $3 - 2 = 1$   | $1 - 1 = 0$   |

$$AWT = ( 2 + 0 + 0 ) / 3 = 0.66 \text{ ms}$$

## Example 2 - SRTF - What if ?

| Pid | AT (ms) | BT (ms) |
|-----|---------|---------|
| P0  | 0       | 10      |
| P1  | 0       | 1       |
| P2  | 0       | 1       |

**Gantt Chart**



| Pid | AT | BT | CT | TAT = CT-AT   | WT = TAT - BT |
|-----|----|----|----|---------------|---------------|
| P0  | 0  | 10 | 12 | $12 - 0 = 12$ | $12 - 10 = 2$ |
| P1  | 0  | 1  | 1  | $1 - 0 = 1$   | $1 - 1 = 0$   |
| P2  | 0  | 1  | 2  | $2 - 0 = 2$   | $2 - 1 = 1$   |

$$AWT = (2 + 0 + 1) / 3 = 1 \text{ ms}$$

## SRTF - The Convoy Effect Problem ?

Example 1:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 1  | 2  |
| BT  | 10 | 1  | 1  |

$$AWT = ( 2 + 0 + 0 ) / 3 = 0.66 \text{ ms}$$

Example 2:

| Pid | P0 | P1 | P2 |
|-----|----|----|----|
| AT  | 0  | 0  | 0  |
| BT  | 10 | 1  | 1  |

$$AWT = ( 2 + 0 + 1 ) / 3 = 1 \text{ ms}$$



The Convoy Effect is completely eliminated by SRTF (Preemptive SJF).

# Where macOS Uses FCFS: :

## 1. 🖨️ Print Queue (Printer Spooler) - What happens:

- You print Document A at 2:00 PM
- Your colleague prints Document B at 2:01 PM
- Another person prints Document C at 2:02 PM

**Order printed:** A → B → C (FCFS)

## 2. 🎧 Audio Processing Pipeline - What happens:

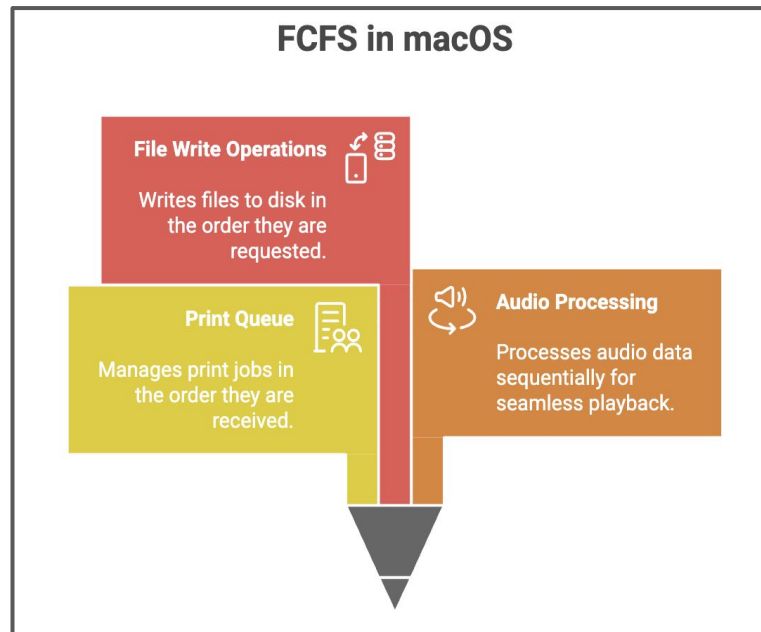
- Audio chunk 1 arrives (0-10ms of song)
- Audio chunk 2 arrives (10-20ms of song)
- Audio chunk 3 arrives (20-30ms of song)

**Processing order:** Chunk 1 → 2 → 3 (FCFS)

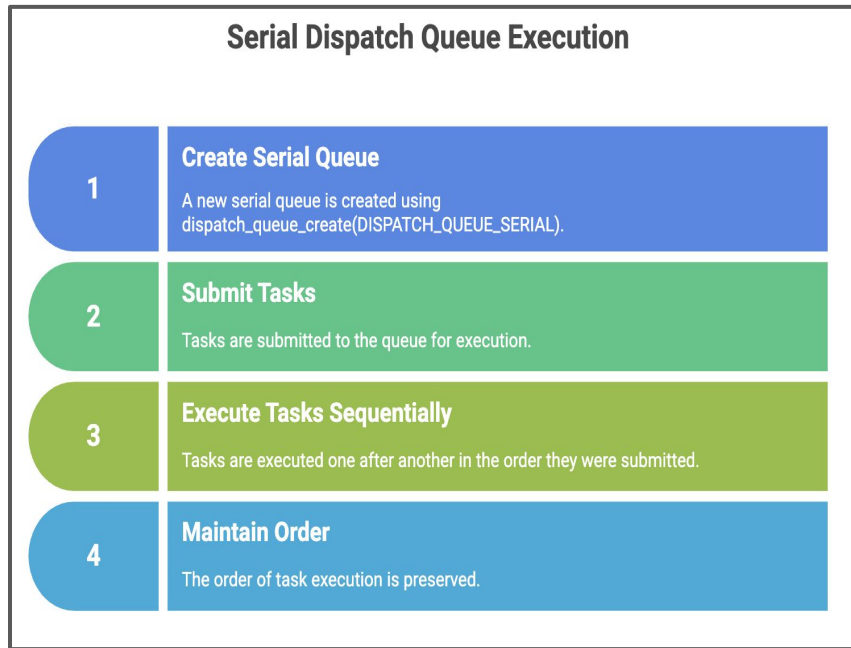
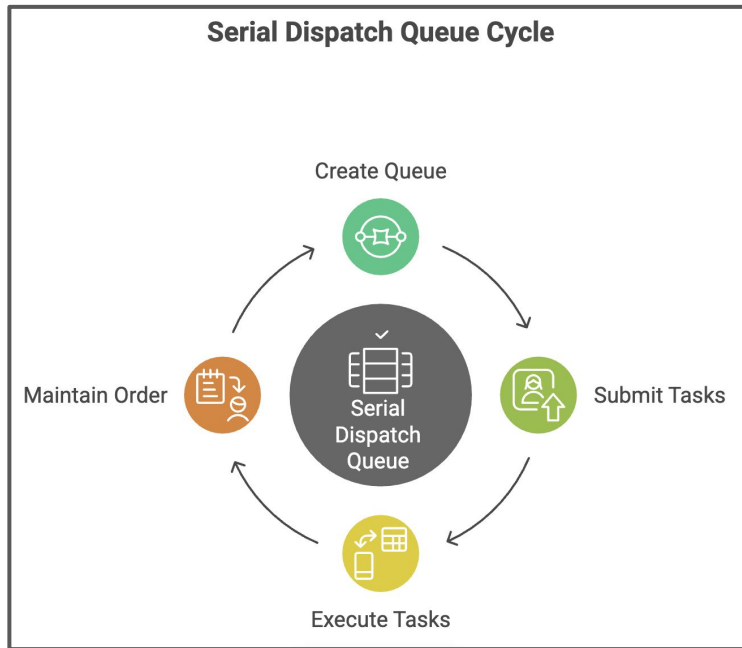
## 3. 💾 File Write Operations (Same File):

- App writes "Hello" to log.txt // Operation 1
- App writes "World" to log.txt // Operation 2
- App writes "!" to log.txt // Operation 3

**Execution order:** Op 1 → Op 2 → Op 3 (FCFS)



# Serial Dispatch Queues (Grand Central Dispatch):



## Where macOS Uses SJF:

1. Quality of Service (QoS): is Apple's way of telling macOS: "How important and urgent is this task?"

Think of it like airport security lines:

-  First Class/Priority = gets through quickly
-  Regular passengers = normal wait
-  Standby passengers = goes last

| QoS Class        | Priority | Assumed Duration |
|------------------|----------|------------------|
| User Interactive | Highest  | Very Short       |
| User Initiated   | High     | Short            |
| Default          | Medium   | Medium           |
| Utility          | Low      | Long             |
| Background       | Lowest   | Very Long        |
| Unspecified      | Medium   | Unknown          |

## Where macOS Uses SJF:

**QoS Classes** = Educated guesses about task duration

- userInteractive → assumed very short → high priority
- background → assumed very long → low priority

**GCD Learning** = Historical execution data

- Tracks how long tasks actually take
- Boosts priority for consistently short tasks
- Prevents long tasks from blocking short ones

**Dynamic Adjustment** = Adapts to real behavior

- If a "short" task takes long → lower its priority
- If a "long" task is quick → boost its priority

**Result:** Gets **80-90% of SJF's benefits** without needing perfect burst time prediction!

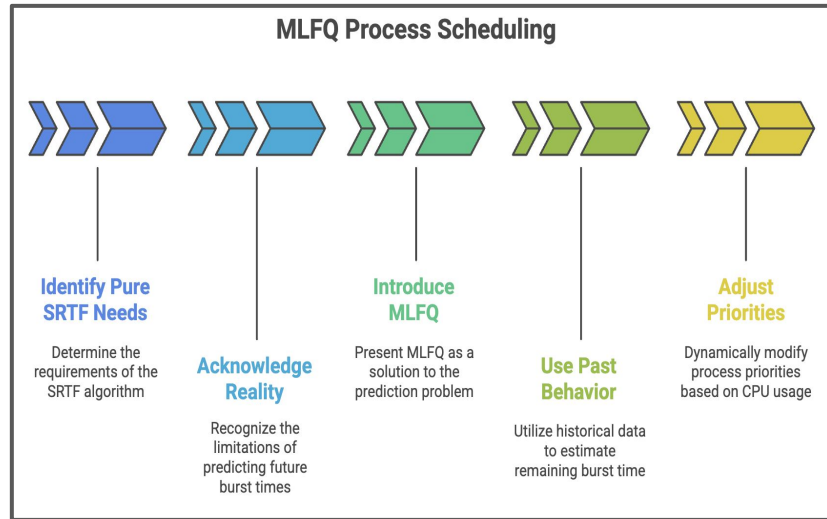
# SRTF in macOS:

## The Problem:

- **Pure SRTF needs:** Exact remaining burst time of every process
- **Reality:** macOS can't predict the future!

## The Solution:

- **MLFQ = Smart guessing system**
- Uses **past behavior** to estimate remaining time
- **Dynamically adjusts** priorities based on CPU usage patterns

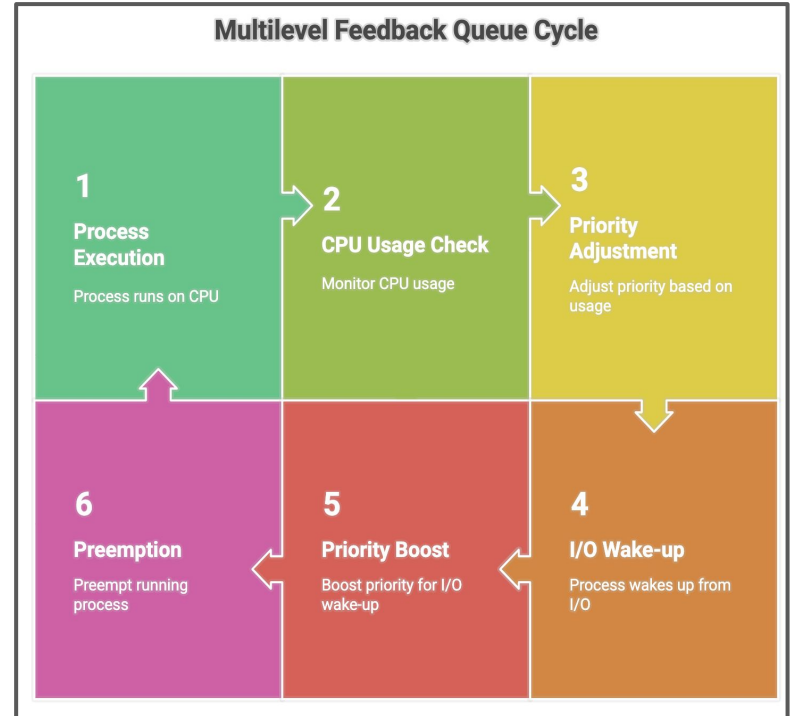




# SRTF in macOS:

## Multilevel Feedback Queue (MLFQ):

- macOS uses a sophisticated hybrid.
- If a process uses too much CPU (Long burst), its priority is lowered (Dynamic Priority).
- If a process wakes up from I/O (Short burst), it gets a priority boost to **preempt** the running CPU hogs.
- This effectively implements SRTF: Short jobs preempt long jobs to keep the Mac feeling "snappy."



## FCFS vs SJF vs SRTF: Complete Comparison:

| Criterion        | FCFS       | SJF      | SRTF         |
|------------------|------------|----------|--------------|
| Preemption       | No         | No       | Yes          |
| Complexity       | Simple     | Moderate | Complex      |
| Avg Wait Time    | Poor       | Optimal  | Most Optimal |
| Response Time    | Variable   | Poor     | Excellent    |
| Starvation Risk  | None       | High     | Very High    |
| Overhead         | Minimal    | Low      | High         |
| Fairness         | High       | Low      | Lowest       |
| Implementation   | Easy       | Medium   | Hard         |
| Burst Prediction | Not needed | Required | Required     |



**Thank You!**